APPLICATION REPORT

HyperVaults

Authentication and cryptography

Architecture, security controls and a verified application demo



Mohamed Yacine Riabi

Yassine ben lazreg

Python FastAPI MongoDB Ethereum Docker Solidity

What HyperVaults does

A wallet-based test-token vault with a separate website identity and security system.

HyperVaults lets a member compare fixed lock offers, deposit HVT, track a funded reward and withdraw after maturity. Rewards come from a sponsor reserve; locking does not create investment yield. The current app uses Python/FastAPI, MongoDB, plain HTML/CSS/JavaScript and Solidity contracts. The selected public network is Sepolia.

We protected the website account without putting wallet private keys in the server. Email and wallet sign-in resolve one account and pass through the same optional MFA gate. The browser wallet still independently authorizes financial transactions. Private notes and MFA secrets are encrypted before being stored.

Verified outcome

54 Python checks and 16 Solidity checks passed, including 512 randomized reward inputs. The real browser demo completed four disposable-chain transactions and 20 responsive page checks. A dependency audit covered 61 runtime packages and found no known advisories.

Reading guide

Report section	Pages	What it explains
System and authentication	3-7	Trust boundaries, credentials, sessions, wallet linking and MFA enrollment.
MFA and cryptography	8-10	Actual setup/sign-in screens and authenticated encryption.
Authorization and transactions	11-12	Roles, ownership, reserves and canonical receipt verification.
Application demo	13-17	Registration through funding, deposits, notes, settlement and administration.
Security assessment	18-21	Implementation steps, negative tests, evidence and remaining limitations.
Live demonstration and sources	22-23	A practical demo script, design justifications and reproducible checks.

Evidence scope. Demo screenshots show the actual app running with disposable accounts and real contracts on an isolated Anvil node. The wallet provider is a test bridge to that node's unlocked accounts. Existing Sepolia contracts were checked read-only; no live funds or account security settings were changed.

Architecture and trust boundaries

The browser presents the workflow; the API enforces access; the contracts enforce financial ownership.

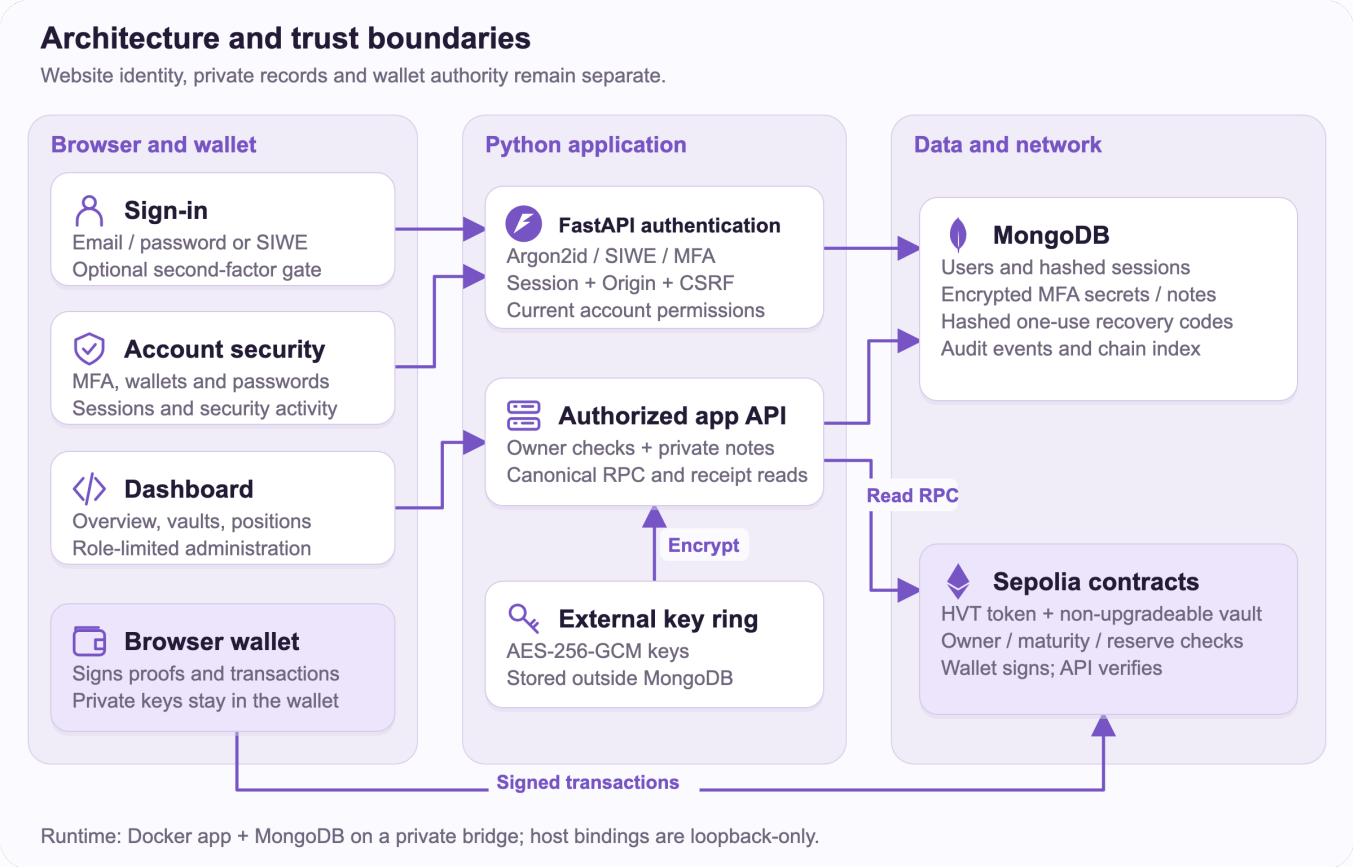


Figure 1. Architecture of the running Sepolia app. The disposable demo follows the same boundaries on Anvil.

Browser. Sign-in, account security and the normal dashboard use separate pages and JavaScript entry points. This keeps authentication controls usable when wallet configuration is unavailable. Connecting a wallet and signing in to the website are separate actions.

Server and data. FastAPI handles identity proofs, sessions, permissions, notes and chain verification. MongoDB stores users, hashed session/reset identifiers, encrypted secrets, audit events and an event index. Unique email and wallet indexes stop silent account merging; expiry indexes clean transient records.

Network and keys. Docker runs the app and MongoDB on a private bridge with localhost-only published ports. The server reads the configured RPC and checks deployment fingerprints. AES keys live outside MongoDB in the external key ring. The browser wallet holds transaction signing authority.

How sign-in is protected

Both primary methods reach the same server-controlled second-factor decision.

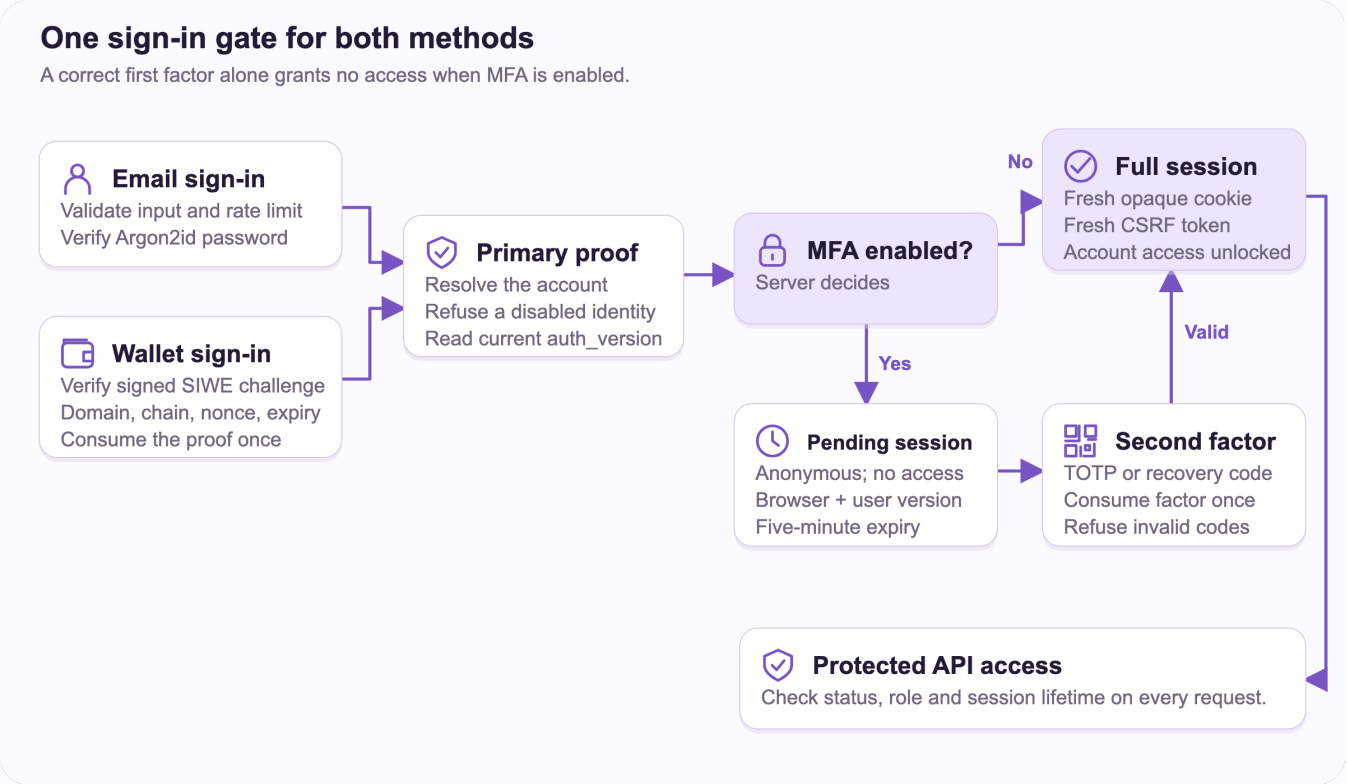


Figure 2. A correct password or wallet signature creates only an anonymous pending session when MFA is enabled.

Email registration. Input schemas reject extra fields and malformed values. New passwords require 12-128 characters; accounts receive the member role on the server. Email syntax is validated and normalized, but email ownership verification is not implemented. The unique index prevents duplicate identities.

Password verification. We use the library's recommended Argon2id password hashing and rehash support. The database contains a salted encoded hash, not a plaintext password or reversible password encryption. Unknown accounts run a dummy hash verification. Invalid or disabled identities receive a generic login failure.

Abuse control

Separate IP and identity budgets limit password and factor guesses. Login permits five identity attempts per minute; invalid shapes are refused before account data is used. These controls reduce guessing, but they do not make passwords or TOTP phishing-resistant.

Design references: OWASP authentication and password storage [1, 3].

How sessions stay controlled

The browser gets an opaque random identifier; account permissions remain on the server.

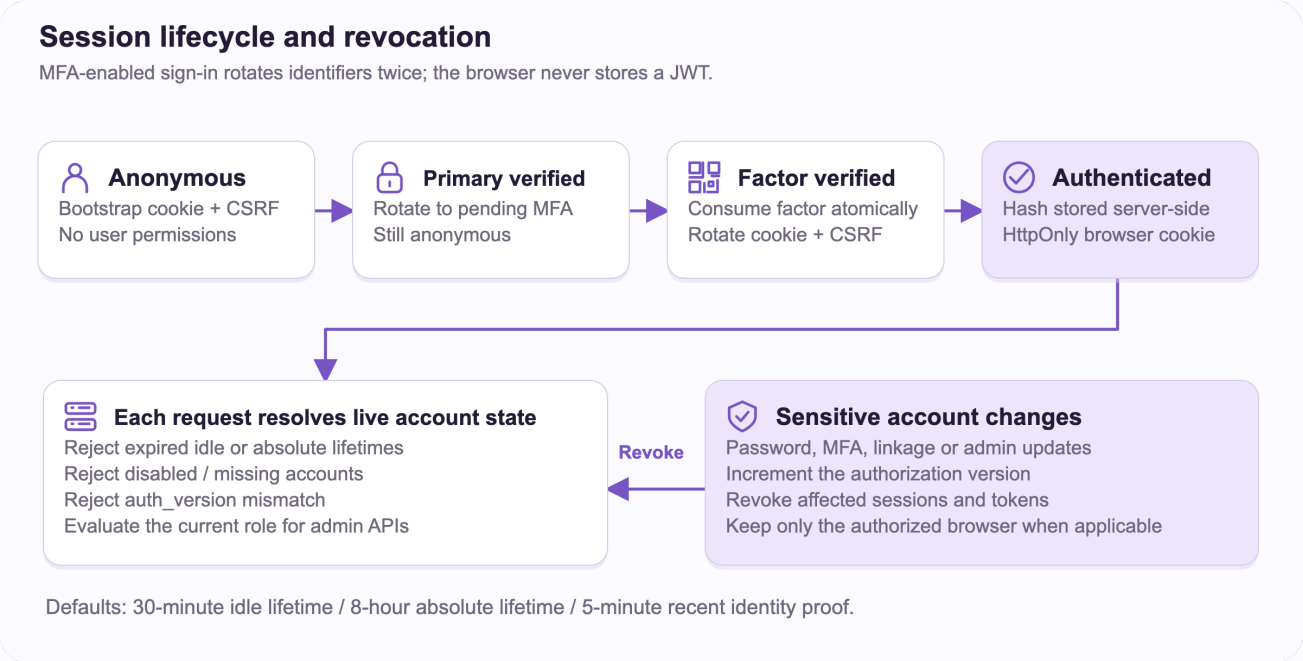


Figure 3. Session rotation, current-account checks and revocation prevent stale identity state from granting access.

Control	Implementation	Why it matters
Token storage	32 random bytes; SHA-256 identifier in MongoDB	A database row does not expose the raw browser session token.
Cookie policy	HttpOnly and SameSite=Strict; Secure and __Host- on HTTPS	Restricts script access and cross-site cookie use.
Lifetime	30-minute idle / 8-hour absolute	Activity cannot extend the fixed absolute expiry.
Reauthentication	Five-minute recent proof for sensitive settings	An old session alone cannot authorize a security change.
Version	Current auth_version and disabled state checked per request	Password, linkage, MFA and admin changes invalidate affected access.
Logout	Delete server session and clear cookie	Replaying the old identifier returns 401.

Mutations also require the exact configured Origin and a matching current CSRF token. CSP blocks external scripts, framing and base-URL changes; no-referrer, nosniff and no-store headers protect the page boundary. HTTPS configuration adds HSTS. Public HTTPS hosting itself was not exercised [2].

How wallet linking works

The wallet proves control of an address without authorizing a token transaction.

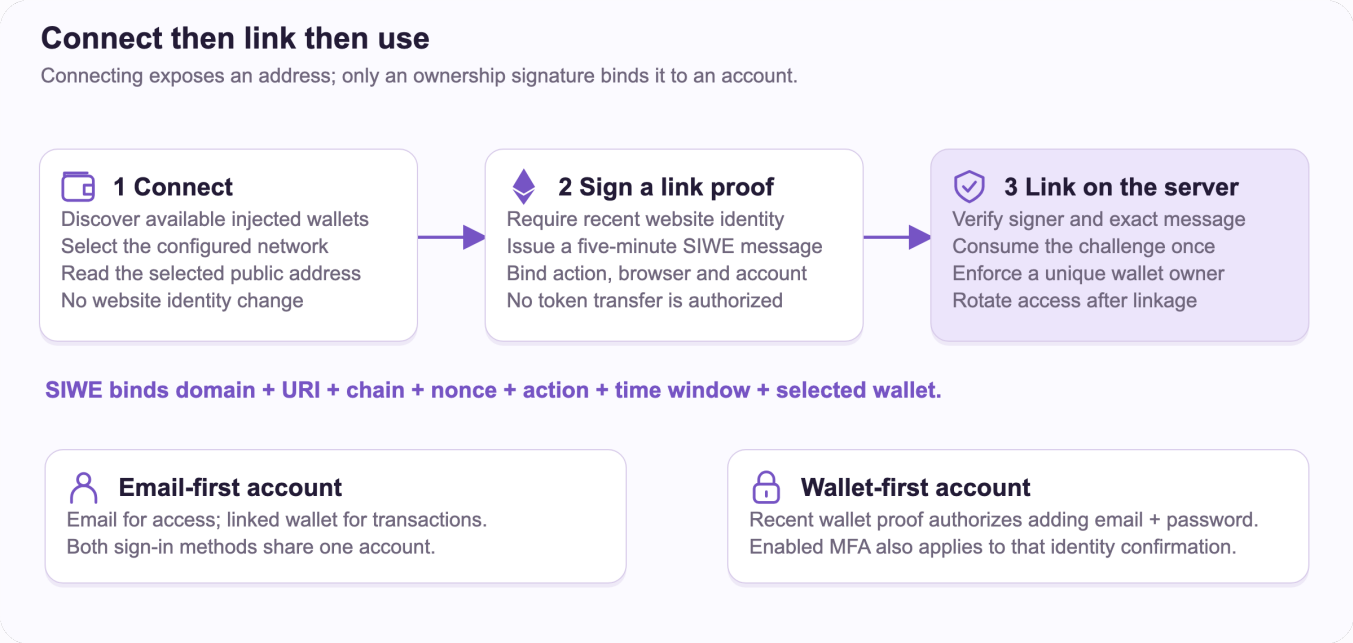


Figure 4. Email-first and wallet-first users keep one identity while the wallet remains the transaction signer.

The server creates a Sign-In with Ethereum message containing its domain, URI, configured chain, nonce, time window and intended action. The stored challenge also binds the browser session, account and authorization version. Verification compares the exact message and recovers the signer before atomically consuming the challenge [5].

Replay and impersonation are refused

Tampering with the message, signing with another wallet, using an expired challenge or repeating a consumed proof fails. Linking requires recent website identity and a signature from the wallet being added. A unique multikey index prevents the same address belonging to two accounts.

Injected key-controlled wallets are supported, including exact EIP-7702 delegation indicators signing with the account's own key. The app rejects contract-wallet signature modes it does not implement. Wallet/provider/account/network changes invalidate the browser's transaction context.

Wallet-only members can add email access after recent linked-wallet reauthentication. Enabled MFA applies to that proof. The server refuses removal of the final sign-in method and limits an account to eight linked wallets.

How MFA enrollment is secured

We activate protection only after the authenticator demonstrates it holds the new secret.

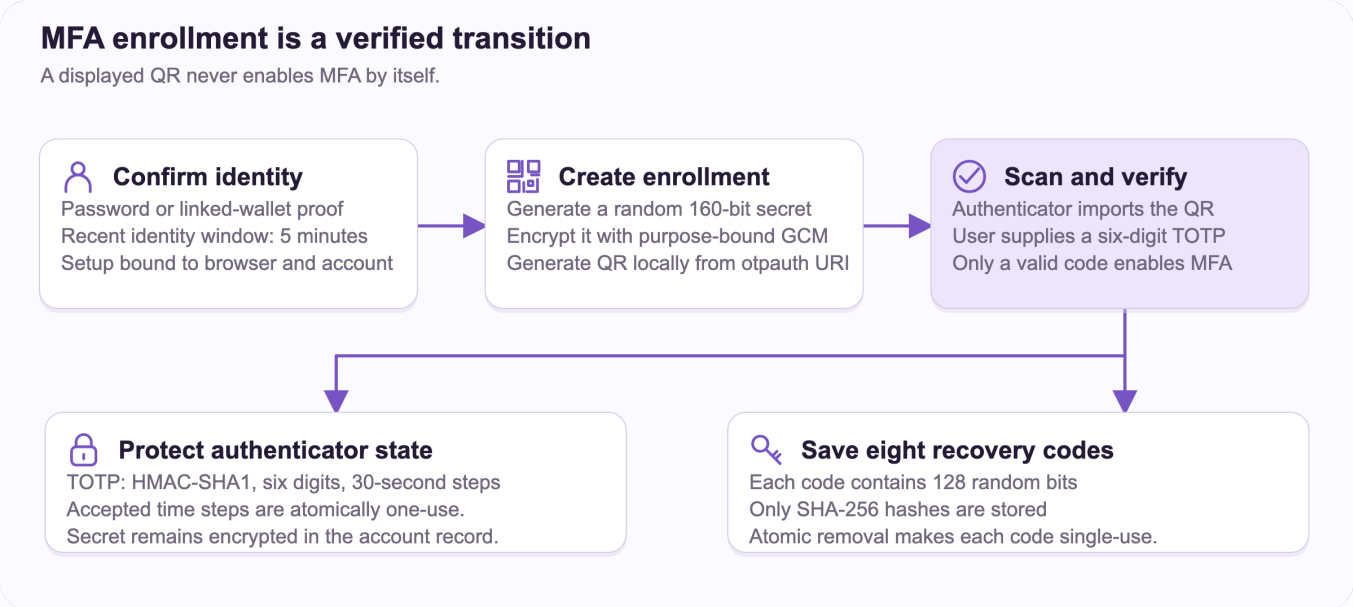


Figure 5. QR setup, encrypted enrollment, verified activation and one-use recovery codes.

Setup first confirms the user's password or linked-wallet identity. A fresh 160-bit secret is encrypted in a browser/account/version-bound enrollment record with a five-minute expiry. The app generates a black-on-white PNG QR locally from the standard otpauth URI; the secret never goes to an external QR service.

The QR response is authenticated, CSRF-protected and marked no-store. The same secret is available under **Can't scan?** as a manual fallback. Canceling, enabling MFA or leaving the page removes the key and image from the page. There is no public image URL and no browser storage of the enrollment secret.

The server checks the first six-digit authenticator code before enabling MFA. TOTP uses HMAC-SHA1, 30-second steps and a one-step drift allowance. Atomic last-step advancement rejects replay. Eight 128-bit recovery codes are displayed once; MongoDB stores only their SHA-256 hashes and atomic removal makes them single-use [6, 8].

Changing MFA needs a full identity proof

Turn off and New recovery codes require password or linked-wallet reauthentication plus an enabled factor. A 60-second confirmation is bound to that named action and consumed once. MFA changes revoke other sessions and stale enrollment/reset state.

Scan then confirm then save

Actual enrollment screens from the disposable demo account.

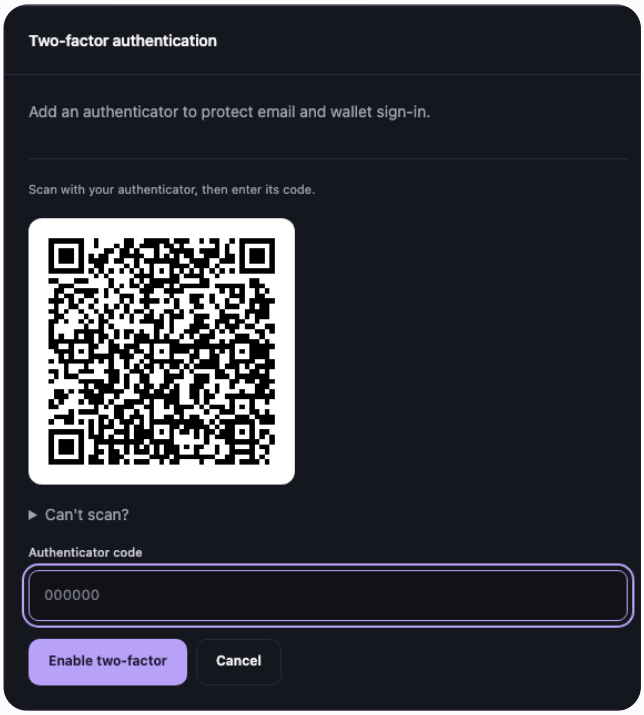


Figure 6. Scan the local QR and enter the code.

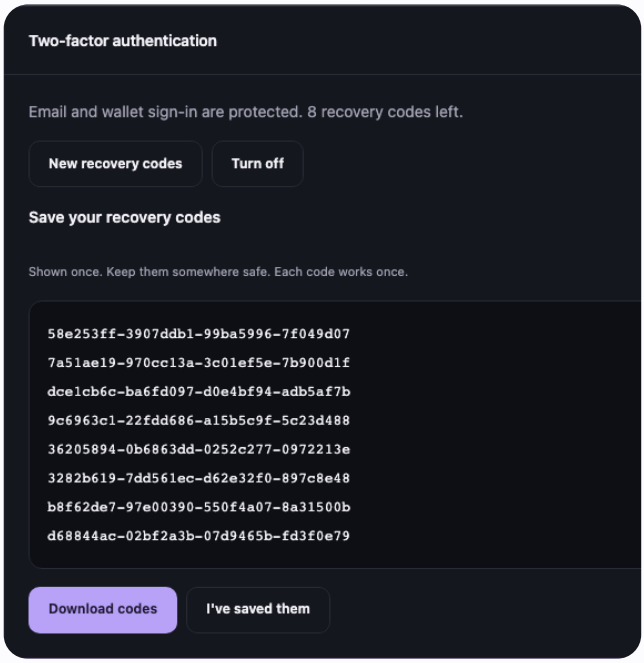


Figure 7. Download the eight one-use backup codes.

1. Open Security. Choose Set up authenticator and confirm the existing identity. Account controls sit outside the dashboard layout; there are no balances or financial actions in this enrollment form.

2. Scan and enter a code. Import the QR into an authenticator, then supply its six-digit code. The UI keeps the manual key collapsed. If the image cannot load, the manual fallback remains usable.

3. Save recovery codes. Protection is now active. Download the codes, store them privately and dismiss the one-time display. New codes replace the old set; each works only once.

QR correctness was tested

An independent reader decoded the actual rendered QR at 1280, 768, 375 and 320-pixel widths. The scanned secret generated a code that enabled MFA through the real browser and API. A physical phone-authenticator scan was not performed.

The pictured enrollment and recovery codes belong to an account deleted when the isolated test ended. They are not credentials for the running Sepolia app.

A primary proof is not enough

The second-factor screen is the only route from pending identity to account privileges.

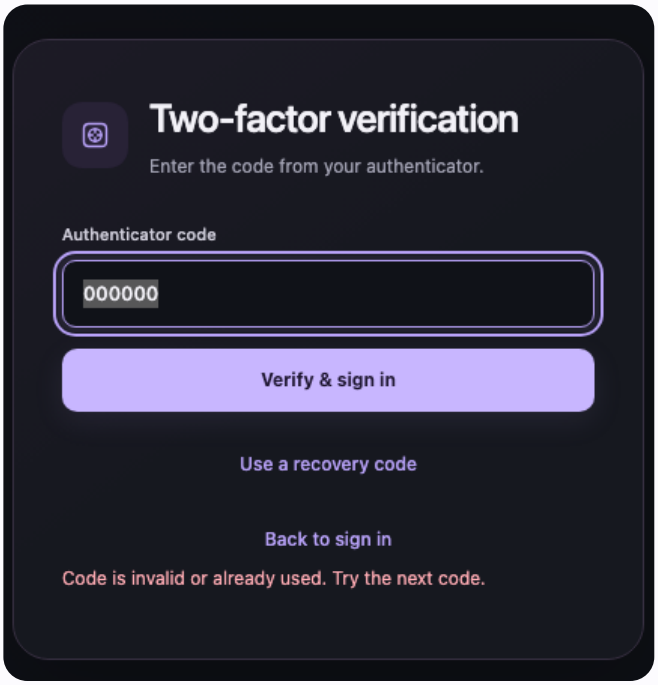


Figure 8. A wrong code is refused without signing the account in.

Email path. A correct password rotated the browser into a pending MFA session. Calling /api/security/sessions from that same browser returned **401** until the second factor succeeded.

Wallet path. The linked-wallet SIWE proof reached the same MFA screen. A recovery code completed that sign-in; the server retained the same user account.

Wrong, expired or already-used codes keep the user in the verification step. Submission guards prevent duplicate sign-ins while the first request is in flight. Use a recovery code offers an explicit alternate factor; Back to sign in cancels the pending flow.

Recovery cannot bypass MFA

A password reset link still needs the enabled second factor, keeps MFA enabled and revokes old access. Wallet ownership cannot disable MFA on its own. Losing both the authenticator and all recovery codes has no automatic website bypass.

MFA protects website access. A wallet can transact directly on-chain without the website, so website MFA cannot replace wallet transaction approval or protect a compromised wallet key.

How stored data is protected

Authenticated encryption binds a record to its intended account and use.

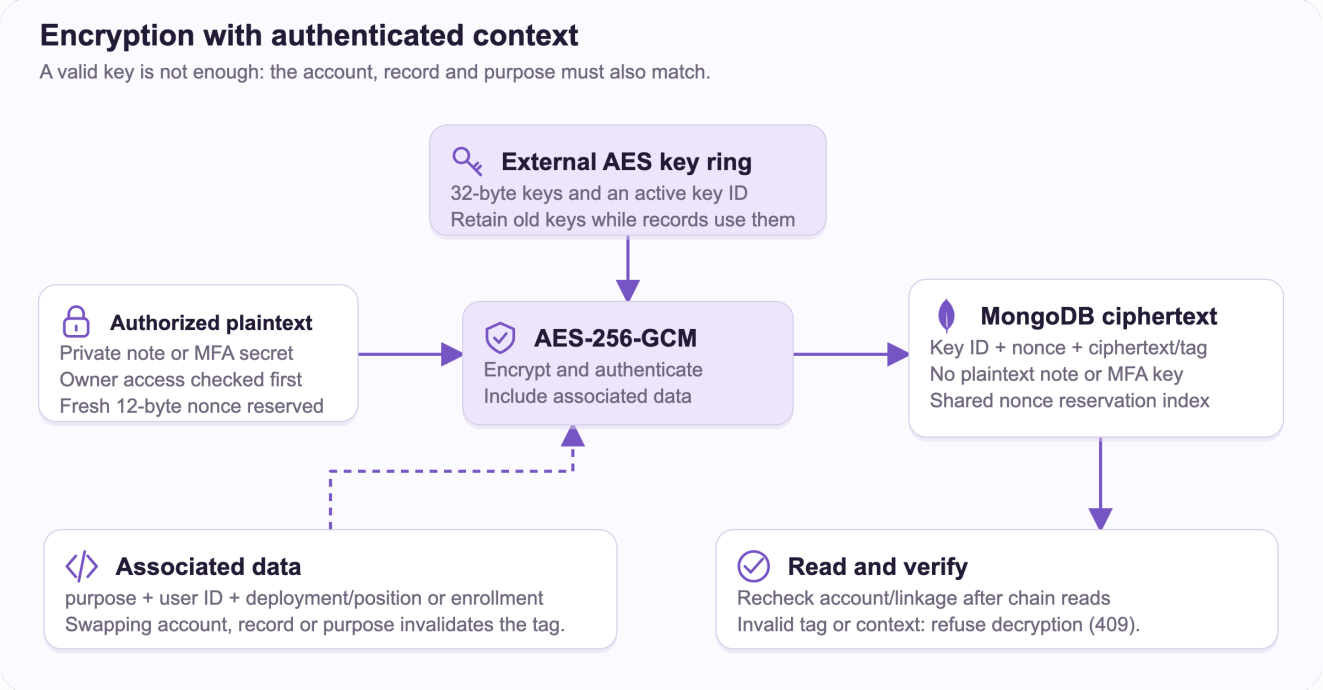


Figure 9. AES-256-GCM protects confidentiality and refuses ciphertext/context tampering.

Private notes and MFA secrets use AES-256-GCM with fresh 12-byte nonces. A shared persistent reservation prevents nonce reuse; exhausted collision retries fail closed. MongoDB stores ciphertext, tag, nonce and key ID. Keys stay outside the database, and old keys must be retained during rotation [4].

Associated data binds purpose, user and record. Notes identify their deployment and position; enrollment uses its own identity. Swapping ciphertext across accounts or purposes invalidates the tag. Authorization is rechecked after slow chain reads before a note is decrypted.

Primitive	What it does here	What it does not do
Argon2id	One-way password hashing with salt and rehash support	It is not reversible password encryption.
AES-256-GCM	Encrypt notes and MFA secrets with integrity and bound context	It is not end-to-end encryption; the server can decrypt.
HMAC-SHA1 TOTP	Generate short-lived authenticator codes	It does not encrypt stored notes or resist phishing by itself.
Ethereum signatures	Prove wallet control and authorize wallet transactions	They do not grant website admin roles.

Who may do what

Roles are checked by the API and ownership is checked independently by Solidity.

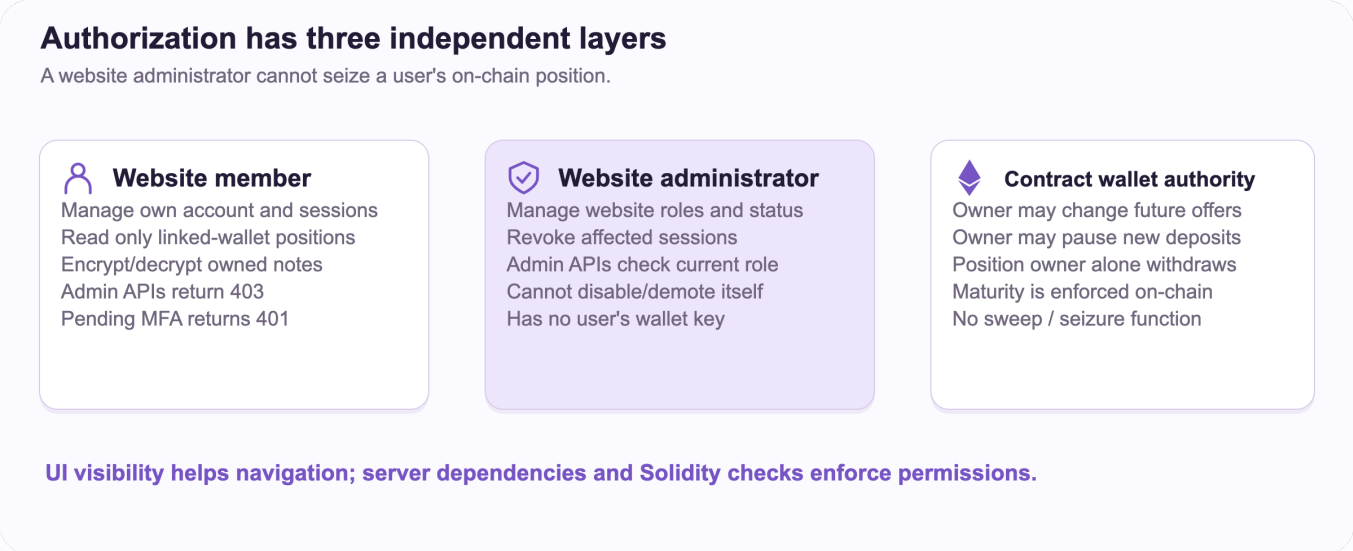


Figure 10. Website member, website administrator and contract authority have distinct permissions.

Action	Required authority	Enforcement
Account settings and sessions	Signed-in account owner	Current user, recent identity and factor checks where required.
Private position note	Account with the position owner's linked wallet	Owner lookup, current linkage/version and GCM authentication.
User roles, disablement and audit	Current website administrator	Server role dependency, self-change restriction and serialized admin updates.
Future offers and deposit pause	Contract owner's wallet	Ownable2Step; website UI additionally requires an admin account.
Matured withdrawal	Original position owner's wallet	On-chain owner, maturity and one-time settlement checks.

A hidden Admin navigation item is a convenience, not the security boundary. Direct admin requests from a member return 403. An incomplete MFA identity receives 401. Guessing another user's session ID cannot revoke it, and knowing a position ID does not disclose its private note.

How financial actions are verified

A submitted transaction must match the network, signer, destination and resulting contract event.

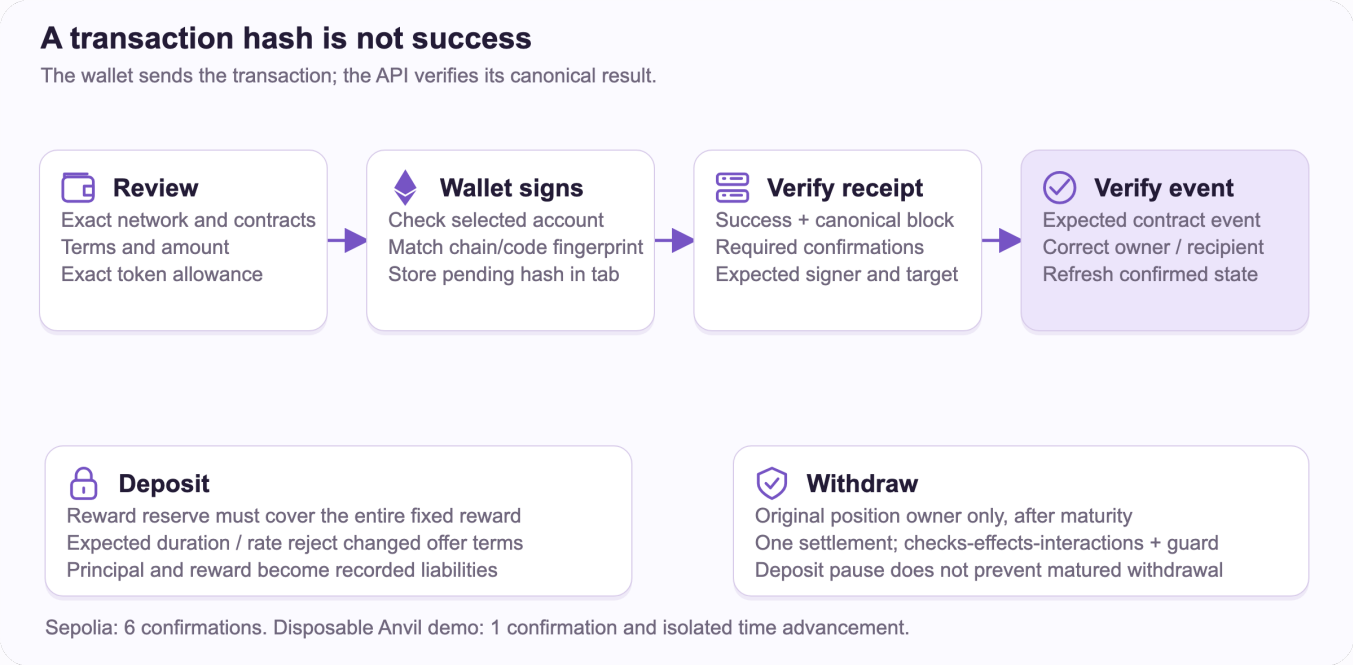


Figure 11. The reviewed action becomes confirmed only after canonical receipt and event verification.

Before signing, the browser checks the selected wallet and chain plus the deployment anchor and contract code hashes. Anvil requires the recorded genesis; Sepolia uses the recorded deployment block. Mainnet is rejected. Account, provider or network changes invalidate the prepared context.

The wallet signs the transaction. The backend holds no wallet private keys and has no transaction-signing endpoint. It checks receipt status, canonical block hash, configured confirmation depth, signer, expected target and matching event fields. Sepolia requires six confirmations; a failed or unavailable verification cannot count as success.

Contract safety rules

The entire fixed reward must already be funded at deposit. Reviewed duration/rate prevent silent term changes. Existing positions keep their terms. Withdrawals require the original owner and maturity, settle once and stay available while new deposits are paused. SafeERC20, exact balance deltas and ReentrancyGuard support these checks [7].

Reward calculation uses integer token base units and floors once: principal × APR basis points × duration / (10,000 × 365 days). Sponsor funding cannot be recovered, and the non-upgradeable vault has no seizure, sweep or early-withdrawal function.

Create an account and link a wallet

Email access starts the website session; the signed wallet link enables the financial workflow.

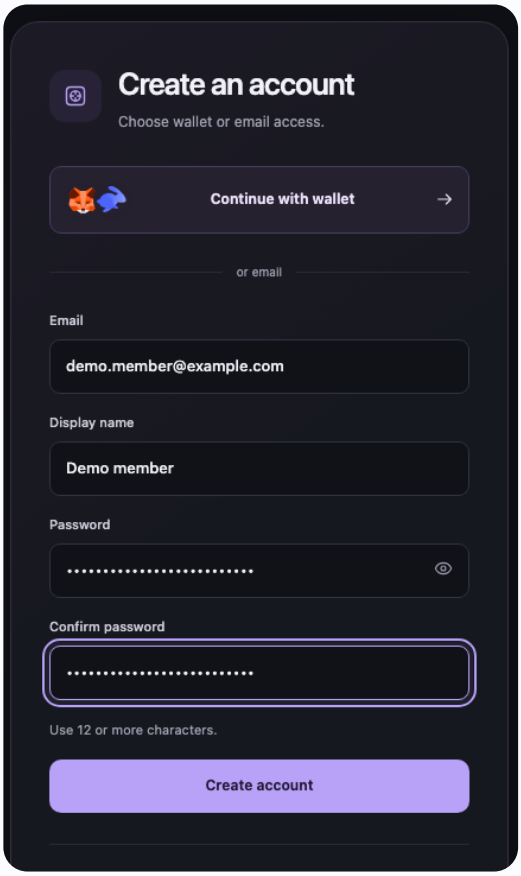


Figure 12. Registration collects a display name, email and matching passwords.

1. Register. The browser submitted the disposable member through the real API. The server assigned the member role and stored an Argon2id password hash. It rotated the session after success.

2. Open the dashboard. Email sign-in creates website access. The account initially has no linked wallet. The dashboard offers a short Connect, Link, Ready guide.

3. Connect. Choose an available wallet. Reading its address prepares the link step; it has not yet changed account ownership.

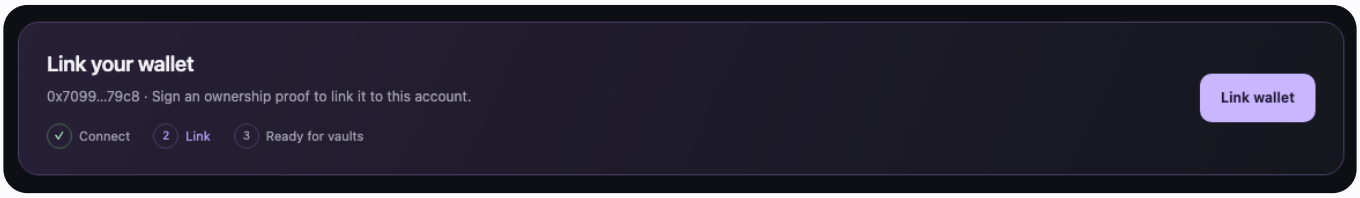


Figure 13. Connecting shows Link wallet; it has not yet changed account ownership.

Connect discovered the test provider and selected the isolated network. The account still had no linked wallets. Link wallet then obtained and verified a real SIWE signature from the disposable node's unlocked account; only that proof attached the address. The onboarding guide disappeared after success.

The provider is a testing bridge, not a live MetaMask or Rabby extension. The actual app's wallet discovery, signature verification and account-linking code executed unchanged.

Review a funded deposit

The user sees the offer and exact amounts before the wallet sends approval and deposit transactions.

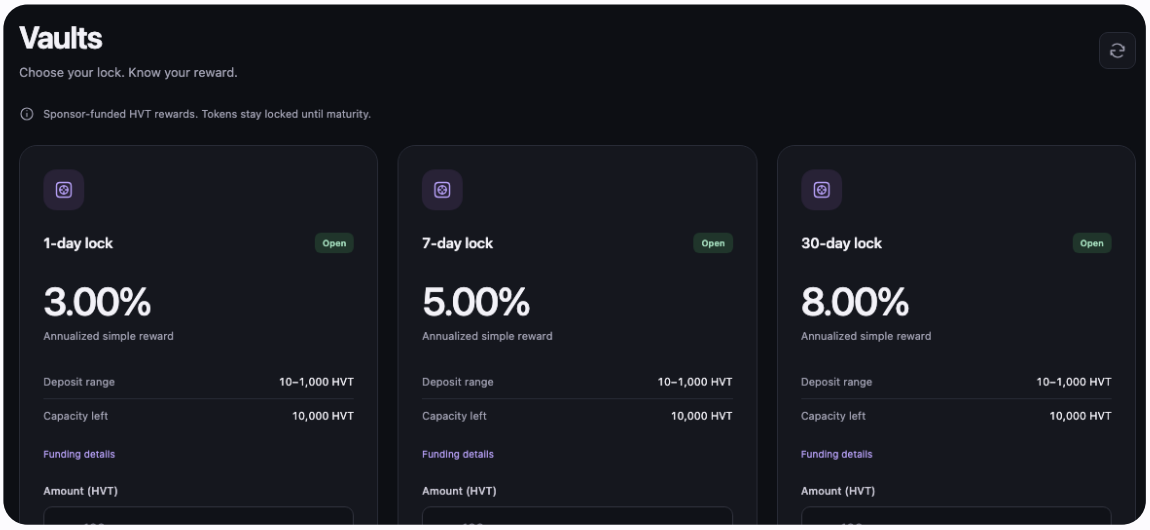


Figure 14. Three open offers in the isolated demo.

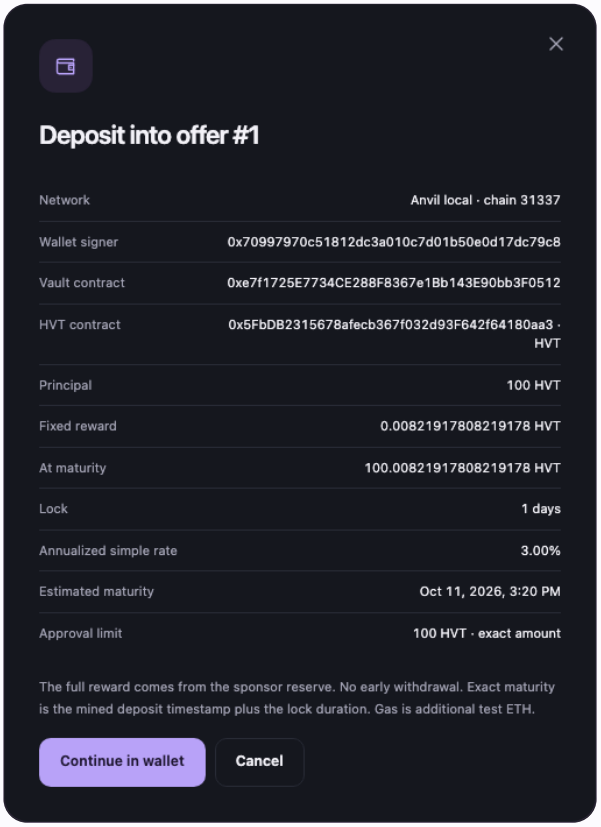


Figure 15. Review lists signer, contracts, principal and fixed reward.

Get HVT. The demo claimed 1,000 test tokens through the browser. The receipt and faucet event were verified by the API.

Preview 100 HVT. The selected one-day demo offer used a 3% annualized simple rate. The reserve covered its fixed reward. The preview explicitly listed both contracts and the selected signer.

Confirm in the wallet. The browser set an exact 100 HVT allowance, then deposited. The consumed allowance returned to zero. Both successful transactions passed receipt and event checks.

The fixture's offer terms are demo inputs; they are not a promise about live investment returns. HVT has no price and is an educational test token.

Track a position and save a note

The dashboard reads contract state while private text is protected by the account API.

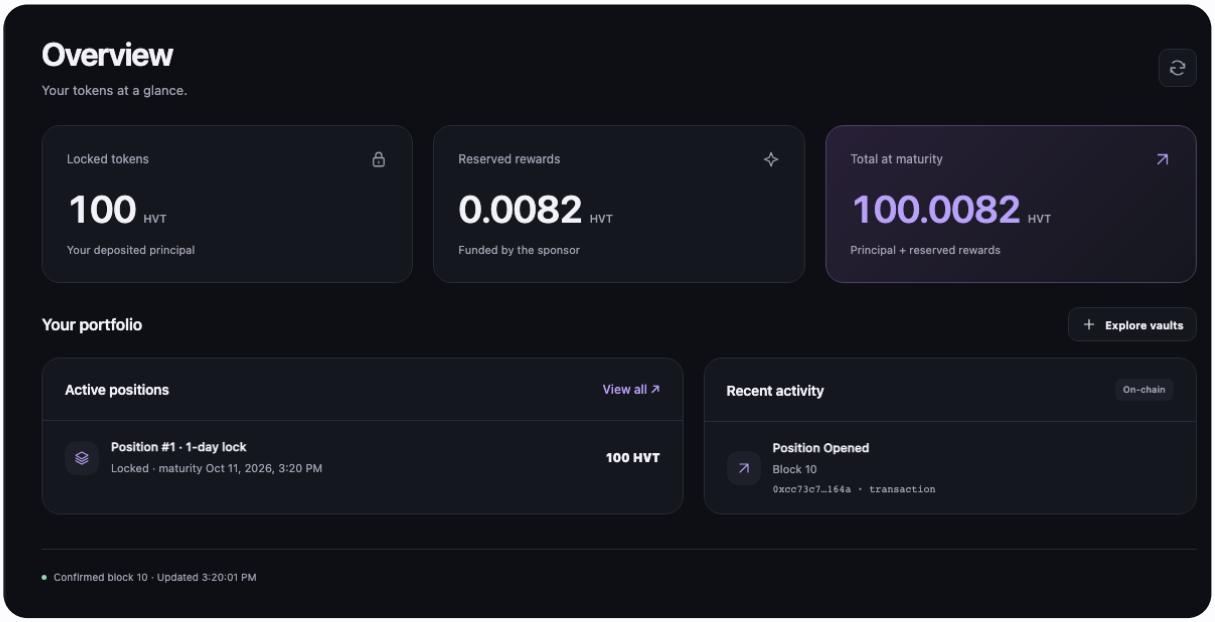


Figure 16. Confirmed state shows 100 HVT principal and its sponsor-reserved reward.

The overview displayed 100 HVT locked and approximately 0.0082 HVT reserved for the one-day offer. Exact amounts remain available in the position details. The withdrawal control was disabled before on-chain maturity; clicking a different website role could not change that rule.

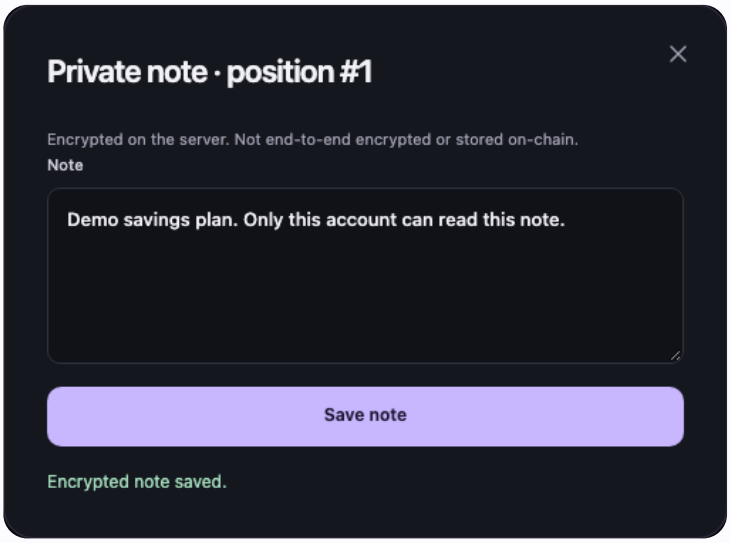


Figure 17. The real API acknowledged encrypted storage of the account's note.

The browser saved the note through the real API. MongoDB contained ciphertext, not the note text. Adversarial tests refused cross-account reads/writes, modified ciphertext and swapped context. Authorized server code can decrypt the note; this is storage encryption.

Withdraw after maturity

The original wallet settles once, including while new deposits are paused.

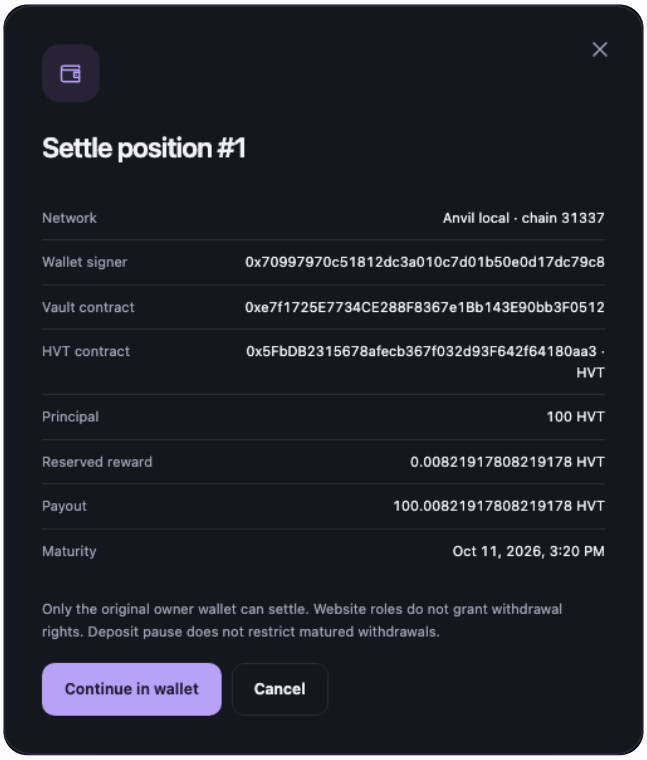


Figure 18. The owner reviews principal, reserved reward and payout.

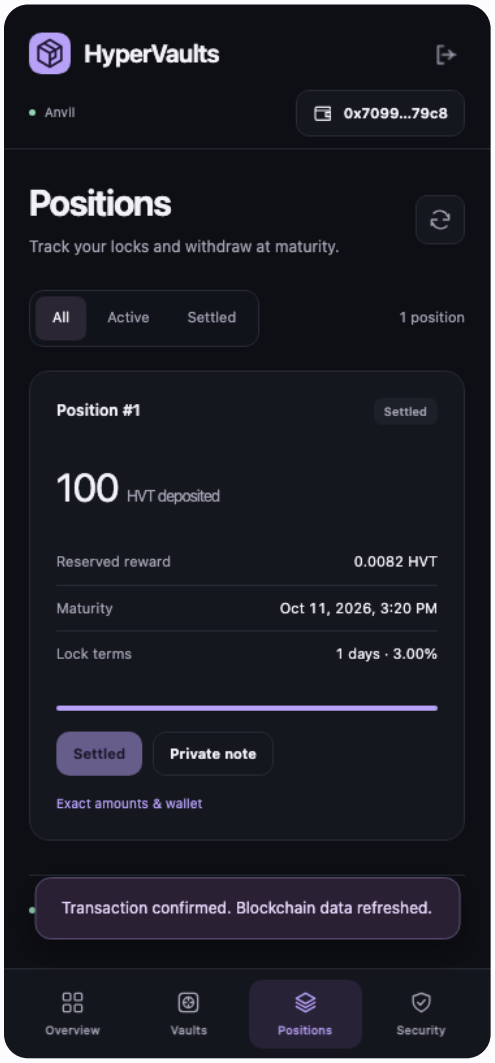


Figure 19. The settled position also renders on a narrow screen.

To test the full lifecycle without a real waiting period, the harness advanced time only on its disposable Anvil node. It paused new deposits, reached the one-day position's maturity and submitted withdrawal from the original owner wallet. No public-network time manipulation or Sepolia withdrawal was attempted.

Settlement checks passed

The backend verified the successful PositionSettled event. Outstanding principal and committed rewards returned to zero, and the owner received principal plus the reserved reward. Contract tests separately rejected early, repeated and other-wallet withdrawals.

Administration stays separate

Website account controls and contract-owner controls use different authority checks.

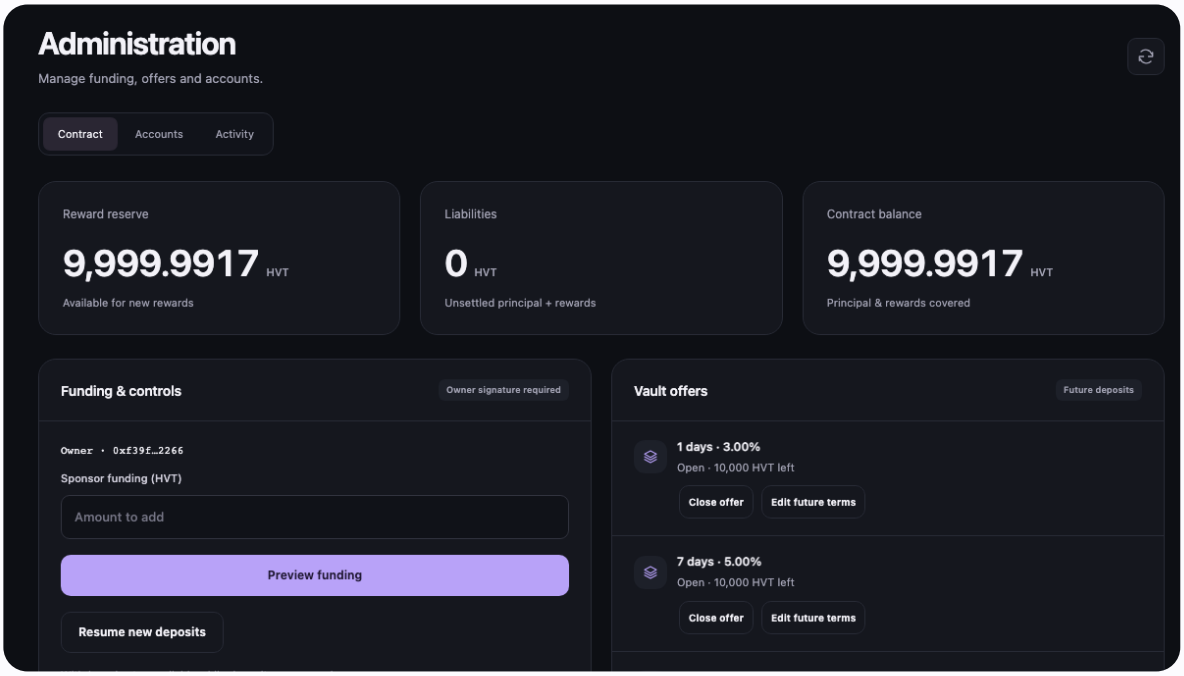


Figure 20. A disposable website admin can inspect coverage without being the contract owner.

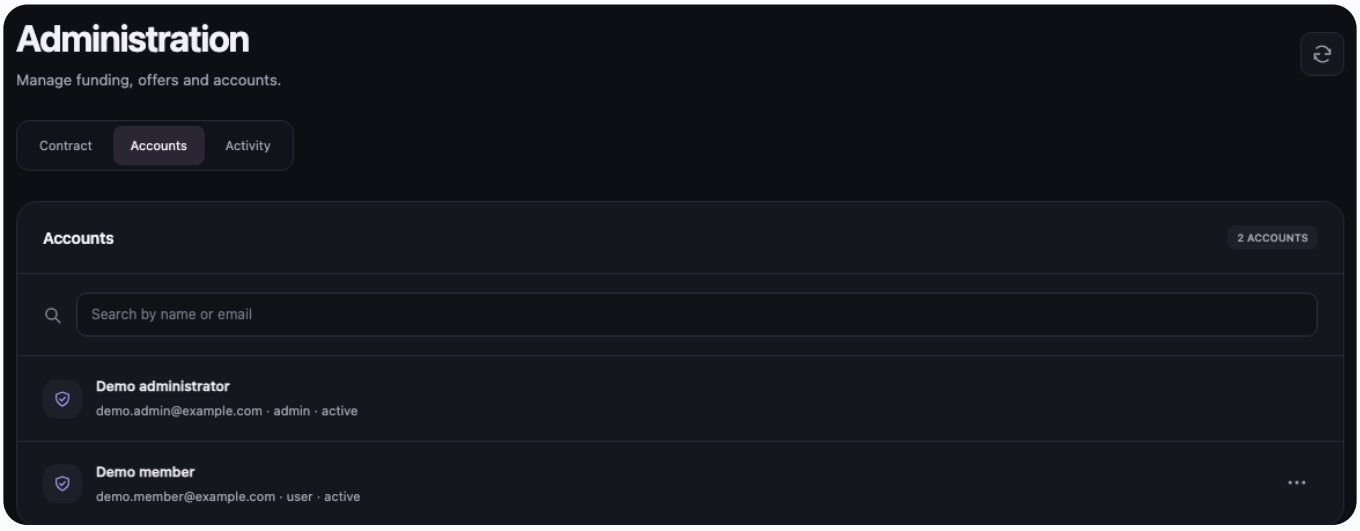


Figure 21. The account directory is available only through administrator-protected APIs.

The harness bootstrapped a separate administrator only inside the disposable database. Contract controls still required the owner's wallet, which that admin did not hold. Member requests to the same admin endpoints returned 403. Disabling an account or changing its access version revoked affected sessions.

No real user role, password, MFA state or linked wallet was edited to produce these screenshots. Administration is protected by current role and recent identity; mandatory MFA for administrators is a remaining improvement.

The steps we took

Security decisions were implemented as explicit boundaries, then checked against misuse cases.

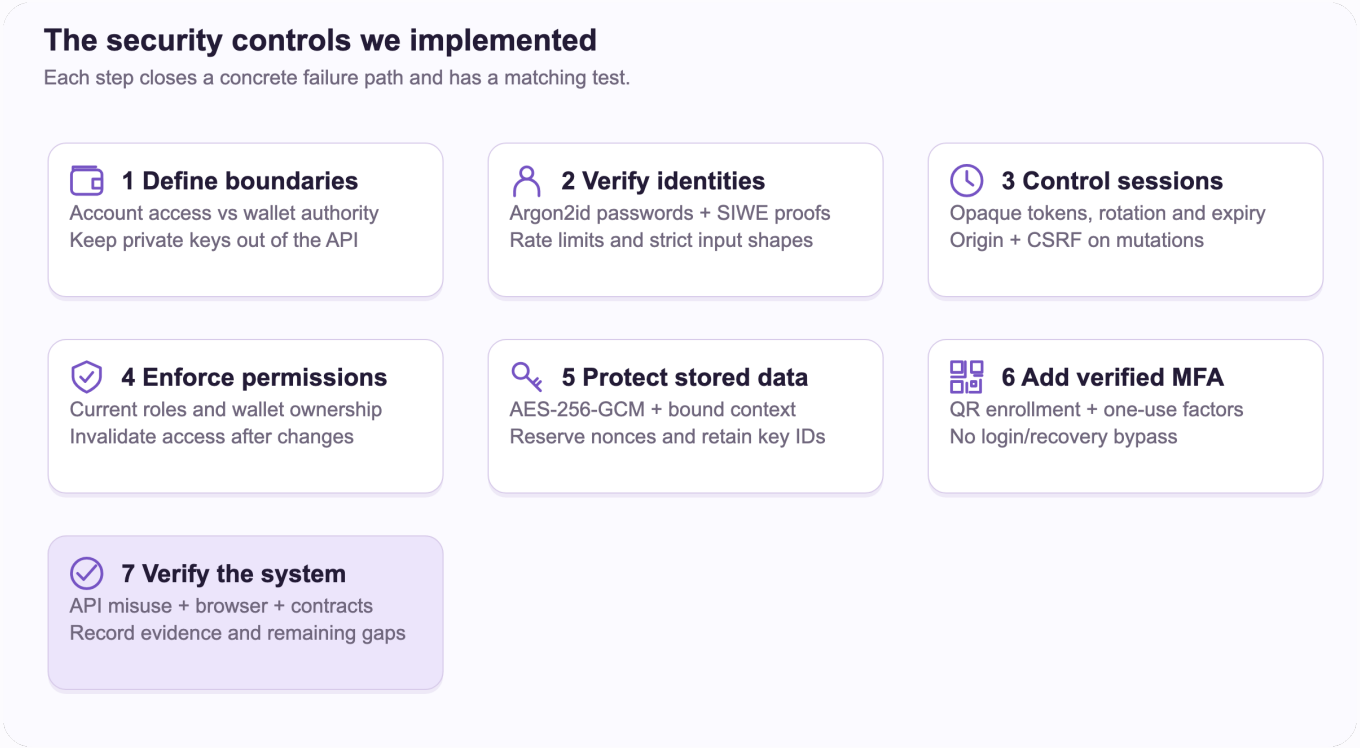


Figure 22. Seven implementation steps connect the architecture to concrete security controls.

Threat	Control we applied	Evidence
Password theft from stored data	Argon2id salted password hash	Hash prefix and plaintext absence checked.
Session fixation or stale access	Rotation, expiry, live status/version checks	Old cookie replay and revoked access refused.
Cross-site state changes	Strict Origin + current CSRF token	Wrong/missing origin or token returns 403.
Identity impersonation or proof replay	Bound SIWE message and atomic consumption	Wrong signer, tampering and repeat proof refused.
Unauthorized reads or tampered storage	Ownership checks + context-bound AES-GCM	Cross-account note access and tag changes refused.
Primary-factor-only access	Anonymous pending MFA + one-use factors	Protected APIs return 401 before factor completion.

Negative tests and expected results

These cases exercise the server and storage layer directly, beyond what the interface allows.

Security case	Expected result	Observed
Registration tries role=admin, malformed email or short password	Validation refusal; no privileged account	PASS / 422
Unknown account vs wrong password	Same generic login failure	PASS / 401
Repeated incorrect password or MFA guesses	Identity/IP budget limits attempts	PASS / 429
Old rotated cookie; idle/absolute expiry; disabled account	No protected account access	PASS / 401
Missing CSRF, attacker Origin, oversize or non-JSON body	Reject state-changing request	PASS / 403, 413, 415
Member calls admin APIs; other account revokes a session	No privilege or cross-account mutation	PASS / 403, 404
Altered SIWE, wrong signer, expired or replayed challenge	No wallet login/link proof accepted	PASS / 400
Same wallet linked to a second account; final login removed	Unique ownership and access retention	PASS / 409
Primary sign-in with MFA unfinished or expired	Anonymous pending state; no permissions	PASS / 401, 400
Concurrent reuse of TOTP or recovery code	Exactly one factor consumption succeeds	PASS / atomic Mongo update
Wrong MFA action or confirmation after 60 seconds	Settings change refused	PASS / 403
Password recovery without the enabled factor; reused token	No MFA bypass or second reset	PASS / 403, 400
Other-account note, tampered ciphertext, swapped AAD	No plaintext or successful decryption	PASS / 404, 409
Access revoked during chain read; repeated encryption nonce	Refuse stale access or unsafe encryption	PASS / 401, failure

The API suite uses a uniquely named real MongoDB database for each case, not an in-memory database replacement. Cleanup removes only those disposable test databases. Tests also confirm private recovery outbox permissions and key-rotation compatibility.

Results and strength of evidence

Passing checks demonstrate specific behavior; coverage and scanning provide additional diagnostics.

Layer	Result	What was exercised
Python API and algorithms	54 / 54 passed	Passwords, sessions, roles, CSRF, SIWE, MFA, encryption and network/setup guards.
Solidity behavior	16 / 16 passed	Ownership, maturity, reserves, pause, terms, fee-token refusal and settlement.
Reward arithmetic fuzzing	512 inputs passed	Bounded randomized principal, duration and rate; exact integer-floor result.
Browser journeys	Passed; no page errors	Registration, linking, faucet, exact approval, deposit, note, withdrawal and both MFA logins.
Responsive layout	20 checks passed	Five pages at 1440, 768, 375 and 320 pixels; no horizontal overflow.
Runtime dependencies	61 packages / 0 known advisories	Pinned non-development dependency set checked with pip-audit.
Running Sepolia app	Read-only checks passed	Health, sign-in/out, CSRF refusal, security APIs, 3 offers and reserve coverage.

Coverage is not a security guarantee

Combined Python branch coverage for auth/, webapp/ and the Sepolia setup script is 77%. Access dependencies are 100%; MFA is 93%; the authorized app routes are 95%. Solidity coverage reports 90.65% lines for the vault, with lower branch coverage and instrumentation anchor warnings. Those diagnostics do not replace the asserted behavior tests.

The browser financial demo used four genuine disposable-chain transaction receipts. The real Sepolia verification used six-confirmation configuration and read-only current state. Physical wallet-extension approval and a phone-authenticator scan were not replayed for this report.

Recovery and operating boundaries

Preserve the state needed to verify data and retain access to encrypted records.

Account recovery. Reset tokens are random, hashed, expire after 15 minutes and are consumed once. The link carries its token in a URL fragment to avoid putting it in HTTP request URLs and referrers. Development messages use a private local outbox; production code uses SMTP with STARTTLS. Enabled MFA stays required during password reset.

Backups. Preserve the database, external key ring and deployment manifests together. Keep older AES key IDs until all records using them have been migrated. Restoring ciphertext without its key, or replacing a local chain while retaining an old fingerprint, cannot restore valid access to that data.

Chain outages. Password and MFA account controls remain usable when wallet configuration is unavailable. Financial actions require verified chain state and stop on mismatched fingerprints or unsupported networks. A position owner can use the contract directly for a matured withdrawal even if the website account is disabled.

Remaining boundary	Current status and next action
Administrator MFA	MFA is optional for all accounts. Require it for privileged accounts before public use.
Email assurance	Syntax and uniqueness are checked. Add email ownership verification and verify real SMTP delivery.
Public hosting	Loopback deployment tested. Validate public HTTPS, certificate configuration, database access and backup restore.
Password policy	12-character minimum; MFA is optional. Consider a stronger password-only policy and compromised-password screening.
Phishing resistance	Passwords and TOTP can be phished. A future passkey/WebAuthn option would strengthen account access.
Independent assessment	No independent audit, Burp engagement or production load/security acceptance was performed.

HVT is unpriced and the contracts are unaudited. Test success does not establish readiness for real funds or production. Public RPC responsiveness also remains an operating dependency.

A clear live demo and justification

Show each boundary working, then explain the decision that enforces it.

Demo step	Show	Explain
1 Register and sign in	Create a member; inspect a generic failed login	The server hashes the password and assigns the role.
2 Connect and link	Address appears first; signature links it	Connection alone does not change website ownership.
3 Activate MFA	Confirm identity, scan QR, verify code, save backups	Activation requires proof of the new secret; backups are one-use.
4 Challenge sign-in	Password succeeds; protected API still returns 401	The pending session is anonymous until the factor passes.
5 Review a deposit	Exact terms, network, signer and token allowance	The wallet signs; a hash alone does not establish completion.
6 Save a private note	Readable UI text; encrypted database record	Account/position/purpose-bound GCM rejects swaps and tampering.
7 Settle and inspect roles	Owner withdrawal at maturity; member admin request refused	Contract ownership, account ownership and website roles are independent.

Design decisions to defend

Opaque sessions. Server-side state supports immediate revocation and current account checks. **Argon2id.** A memory-hard password hash avoids plaintext/reversible password storage. **Bound signatures.** SIWE proves a specific wallet action in a specific browser/time context. **AES-GCM.** Encryption and authentication detect stored-data tampering. **Anonymous MFA pending state.** No application privilege exists before all required factors pass.

Keep the demonstration reproducible

Use disposable test accounts and an isolated Anvil node for deposits, time advancement and withdrawal. Use the running Sepolia app for read-only deployment and account-screen verification. Never paste a real wallet private key or seed phrase into a script, app or report.

Sources and reproducible checks

The report links the implemented controls to local source, tests and primary specifications.

Local source. auth/services.py and auth/routes.py implement identity/session controls; auth/mfa.py and auth/mfa_routes.py implement MFA; auth/crypto.py and webapp/routes.py protect notes. webapp/blockchain.py verifies canonical chain reads/receipts. smartcontract/src/ contains the token and vault. tests/test_security.py and smartcontract/test/HyperVaults.t.sol add the broader security checks.

Check	Run from the repository
API and algorithm suite	HV_AUTH_TEST_MONGO_URI=mongodb://127.0.0.1:27018 uv run python -m unittest discover -s tests -v (set the variable for the same command invocation)
Contract suite	.tools/bin/forge test --root smartcontract --fuzz-runs 512 -vv
Real browser app demo	With HV_AUTH_TEST_MONGO_URI set: uv run --with playwright python -m tests.check_app_e2e
Focused browser checks	uv run --with playwright python tests/check_auth_ui.py uv run --with playwright python tests/check_account_ui.py

[1] Authentication - OWASP Cheat Sheet Series

[2] Session management - OWASP Cheat Sheet Series

[3] Password storage - OWASP Cheat Sheet Series

[4] Cryptographic storage - OWASP Cheat Sheet Series

[5] Sign-In with Ethereum ERC-4361

[6] Time-based one-time passwords RFC 6238

[7] OpenZeppelin access control

[8] Local QR generation with Segno

[9] Technology logo artwork

Technology marks are used to identify the actual stack; they do not imply endorsement. SVG artwork comes from Simple Icons, with its CC0 artwork license and each brand's separate trademark guidelines. HyperVaults artwork comes from the repository. All nine report diagrams remain editable SVG files.